

Commission & Consolidator Payouts

Module specification for CRM integration

A. Appel and Co - sub-agent under Main St. Travel (IATA 33535983)

This module answers one question on every ticket: **how much will the host agency's cheque be, and is it right.** It reads the ticket, finds the contract that governs it, computes the figure, and pre-fills the commission field. Once a week the host's statement is uploaded and the module reconciles what was actually paid against what was owed.

It is built as a **pure function**: text in, JSON out. No database calls, no clock, no network. The same ticket always produces the same number, and the same code runs inside the CRM, in a queue worker, in an Edge Function, and in tests.

Scope for the first build

EL AL (LY) under Main St. Travel only. That is where the volume is, and one airline under one host is enough to prove the whole path end to end. Everything below is designed to take more hosts and more airlines without a rewrite - the data model already keys on them - but do not build ten before one works.

1 - The three figures

This is the heart of the module. Every ticket produces three numbers, in this order, and they must not be collapsed into one.

#	Figure	What it is
1	Commission	What the airline pays, less the host's share. Zero on a net or bulk fare, and zero on a published fare that fails a contract condition.
2	Fees	What the host charges: per exchange, per refund, on a net fare, on a ticket that earned nothing. Always negative.
3	Markup	On a net fare, the agent's own margin - selling fare less net fare. It is the agent's commission on that ticket, self-set inside a ceiling the airline permits.

The cheque

$$\text{cheque} = \text{commission} + \text{markup} - \text{fees}$$

The markup belongs in the cheque. The whole ticket, markup included, is collected under the host's accreditation and settles to them - so the markup is not money the agent already holds, it is money the host is holding on the agent's behalf and has to remit. From the agent's side there is one figure, and this is it.

This is the single most expensive thing to get wrong. Reconciling a statement against the COMMISSION alone reports every marked-up bulk ticket as paid-where-nothing-was-due, and a bulk ticket missing from the statement as agreement rather than a shortfall. On the sample batch that is the difference between an expected total of \$89.76 and \$3,245.26 - the gap is markup being silently written off. Reconcile against the cheque.

Worked, on real tickets

Ticket	Fare	Base	Commission	Fee	Markup	Cheque
bulk business	net	12,378.75	0.00	-123.79	2,475.75	2,351.96
marked-up economy	net	3,398.75	0.00	-33.99	679.75	645.76
published S	published	1,496.00	0.00	-10.00	0.00	-10.00
exchange	net	3,707.00	0.00	-25.00	0.00	-25.00

Note the third row: a ticket can be **negative**. It earned no commission and still cost a fee. That is a real outcome, not an error, and the UI has to show it.

2 - Net fares: the rule that governs them

On a net fare the airline files a fare it will accept and lets the agent sell above it. The agent's margin is their commission on that ticket. The host's fee for such a ticket follows one principle:

The host cannot be worse off because the agent chose to issue net. Whatever the host would have earned had the same fare gone out published with commission, they earn. The agent's choice of how to take their margin is theirs, and it is not allowed to cost the host anything.

So the host's fee on a net fare is **the higher of** the point they keep on a published fare, and a flat figure by cabin. The cabin figure is a FLOOR, not the fee - it applies only to fares small enough that a point of them comes to very little.

$$\text{net fare fee} = \max(\text{published-fare point} \times \text{base fare}, \text{cabin floor})$$

12,378.75 business: 1% = 123.79 floor 50 -> 123.79
900.00 economy: 1% = 9.00 floor 15 -> 15.00

Reading the cabin figure as the fee rather than as a floor understates the host on every large fare - \$50 where \$123.79 is owed on the sample business ticket - and would have the reconciler reporting the host's own correct deduction as an unexplained one.

The markup ceiling

The airline permits a markup up to a limit, which varies by fare. Where that limit is known and configured, the module reports the markup as a percentage and the **headroom** left under it. Headroom matters because it is the one loss no statement can show: marking up 10% where 25% was permitted leaves money behind, nobody short-paid anything, and reconciliation will say AGREES. An unconfigured ceiling reports as **unknown**, never as zero.

Reading the markup from the file

Amadeus records it in the FM element as FM*G*679.75A. Those digits match the commission-amount pattern exactly, so **the *G* must be tested first** - otherwise the agent's own margin is filed as commission the host owes, and every marked-up ticket reports a shortfall. Cross-check it against selling base less net base (KS- less KN-); the two are independent and must agree.

3 - Finding the contract

Every ticket carries the IATA number it was issued under. **The IATA number owns the contract, not the agency** - one host commonly holds several numbers with different rates on the same airline. The MST agreement says so itself: "PNRs created under one of MST's affiliate offices in order to access higher contracted commission levels."

The chain is walked **in order** and stops at the first step that fails. Each step reports which step it was, because they are four different things to do about it.

#	Step	If it fails	What that means
1	Tenant	reject	Never price across tenants
2	IATA on the ticket	no_iata	Nothing to match on
3	Office for that IATA	unknown_iata	A contract to go and get
4	Contract for the airline	no_carrier	A contract to negotiate
5	Ticketing date in window	outside_window	A renewal
6	Clause: route, class, fare type, tour code	NO_RULE	Contract held, nothing covers this

Steps 1-5 are the **fallback**: no contract could be chosen, so no figure is written. Step 6 is different - a contract exists and no clause covers the ticket.

NIL and NO_RULE must never merge. NIL means a clause asserted zero - EL AL forfeits the commission when the mandatory tour code is missing, and that is a settled answer. NO_RULE means nobody has worked it out. Both display as \$0.00; only NIL may be written into a field. Merging them books zero on tickets that were actually owed money, which is the failure this module exists to prevent.

Bootstrapping from the files

discoverFromFiles() groups incoming documents by IATA and carrier and reports what it sees - ticket counts, date ranges, fare types, tour codes, and the rates actually claimed per booking class - flagging each group as configured or not. Use it to populate the contract table from history rather than typing it blind.

It proposes a rate only where **every** ticket in a class agreed on one. Where they disagree it reports the disagreement instead: what an agent claimed is evidence, not a contract, and seeding a table with the more popular of two wrong answers would encode the mistake as policy and then reconcile against it forever.

4 - Data model

Everything is editable, and contracts are uploadable as files. Scoped by tenant throughout.

Table	Key columns
tenant	id, name. Every query below is scoped by tenant_id.
office	id, tenant_id, iata (the join key, unique per tenant), name, agency (label grouping several offices under one host), retains_points, fee_schedule

Table	Key columns
carrier_contract	id, office_id, carrier , issued_from, issued_to, rates (JSON: class -> percent), include_yq, required_tour_code, origin_in, scope, max_markup_percent, markup_basis, files[]
host_agreement	id, office_id, the fee schedule: per-fare-type splits and the fee rows (exchange, refund, net fare by cabin, non-commissionable). Each row carries approved: boolean .
ticket_document	One row per passenger per document: ticket number, type, issue date, carrier, iata, base fare, net fare, taxes, coupons, fare type, tour code, plus the computed commission, fees, markup, cheque and outcome.
statement_row	One row per line of an uploaded statement: ticket number, amount paid, fees withheld, statement date.

Keep the approved flag. Clauses that depend on facts an AIR file does not carry - corporate bookings, promotional net fares, discretionary minimum fees - are encoded but left unapproved, so the engine surfaces them as exposure rather than booking them as certainty. An unapproved clause never spends.

Adding a second host later

Nothing structural changes: a new host is new office rows (one per IATA number) with their own carrier_contract and host_agreement. Resolution already keys on tenant + IATA + carrier + date, so tickets route themselves.

5 - The pre-fill rule

The module returns prefill: a decimal string, or null. **Bind it straight to the field.** The decision about whether a number is safe to write has already been made - the caller does not repeat it.

Outcome	prefill	Meaning
CALCULATED	the amount	A clause matched and produced a figure
NIL	0.00	A clause asserted nothing is due - a settled answer
NO_RULE	null	Contract held, no clause covers this ticket
AMBIGUOUS	null	Two clauses matched equally. Never guessed
INCOMPLETE	null	A clause needed something the document lacked
NO_CONTRACT	null	No contract for this IATA - the fallback
ERROR	null	Could not be priced at all

Two further conditions suppress a pre-fill even on a good outcome:

Condition	Why
The layer being paid must be the layer that settled	On a Zurich-Tel Aviv reissue the carrier layer resolved cleanly (a \$100 clawback) while the sub-agent layer could not resolve its share. Reading one and paying the other offered a confident \$0.00 on an open question.

Condition	Why
No REVIEW flag on the document	REVIEW is raised when two signed contracts disagree - on that same ticket, that the host pays \$100 out of pocket. A figure already flagged for a human is not one to bind to a field.

Every result carries explanation, clause and ruleId. Show the explanation beside the field. A number the agent cannot trace to a line of the contract is one they cannot dispute, which defeats the point.

6 - The weekly statement

Uploaded as CSV or Excel. Each row is matched to a ticket by ticket number and lands in exactly one of eight states. Nothing is dropped: a row matching no ticket, and a ticket on no row, are both findings. **Compare against the cheque, not the commission.**

Status	Severity	Meaning	Action
SHORT_PAID	critical	Paid less than the contract gives	Claim the difference
NOT_ON_STATEMENT	critical	Owed, and the statement omits it	Claim in full
OVER_PAID	warning	Paid more than the contract gives	Expect a clawback
PAID_WHERE_NONE_DUE	warning	Paid on a document owed nothing	Verify before spending
NOT_IN_TICKETS	warning	Paid for a document not in our records	Investigate
DEDUCTION	warning	A fee withheld no agreement covers	Query the fee
AGREES	ok	Matches to the cent	-
CORRECTLY_NIL	ok	Both sides agree nothing is due	-

Excel export

One row per ticket, sorted **worst first** (critical, warning, ok) so what needs attention is at the top. Variance is the money column; negative means underpaid.

Column	Example	Column	Example
Status	SHORT_PAID	Commission	0.00
Ticket	114-7507683179	Fees	-33.99
Passenger	TESTPAX/SAMPLE	Markup	679.75
Type	TKT	Cheque due	645.76
Issue date	2026-08-31	Paid	600.00
Route	JFK-TLV	Variance	-45.76
Class / fare	K / net	Contract	EL AL 2026

Column	Example	Column	Example
Base fare	3398.75	Why	net fare fee, 1% of fare

Add a totals row splitting variance into *short-paid*, *over-paid*, *missing* and *unexplained deductions*. That figure is what the week is worth arguing about. Red fill on negative variance, amber on positive; freeze the header.

7 - API

```
import { priceAirFile, checkStatement, discoverFromFiles }
  from '@commission/sdk'

const result = priceAirFile(airText, {
  config,                // contracts, loaded from your DB per tenant
  tenantId: 'acme',
  view: 'subagent',      // or 'host'
})

for (const doc of result.documents) {
  doc.ticketNumber       // '114-7507683179'
  doc.prefill            // '645.76' or null -> leave the field empty
  doc.outcome            // 'CALCULATED' | 'NIL' | 'NO_RULE' | ...

  doc.subAgentCommission // '0.00'      airline's commission, less host share
  doc.fees               // [{ clause, label, amount }]
  doc.markup            // '679.75'    the agent's own margin
  doc.totalToSubAgent    // '645.76'    THE CHEQUE - reconcile against this
  doc.netToSubAgent      // '-33.99'    commission less fees, excludes markup

  doc.markupPercent      // '25.0000' or null if no ceiling configured
  doc.markupHeadroom     // '0.00' = at the ceiling; negative = over it

  doc.explanation         // show beside the field
  doc.flags              // [{ code: 'REVIEW', message }]
}

checkStatement(statementCsv, airFiles, opts) // weekly reconciliation
discoverFromFiles(airFiles, opts)           // bootstrap contracts
```

All money crosses the boundary as decimal strings - no bigint, no floats, no Date - so the whole response survives JSON.stringify into a column or an HTTP body. An HTTP wrapper is provided (POST /price, POST /statement) with no dependencies, for callers that cannot import JavaScript.

8 - Build order

Phase	Scope	Done when
1	Contract tables + resolution. Read-only: compute and display, write nothing.	Every LY/MST ticket shows a cheque figure and an explanation
2	Pre-fill. Write the number into the field.	Agents stop entering commission by hand on CALCULATED and NIL tickets
3	Statement upload, reconciliation, Excel export.	A week's statement produces a sorted exception list
4	Contract editing and PDF upload in the UI.	A new airline can be added without a deploy
5	Second host agency; then standalone for other agencies.	The engine has no CRM dependency - this is packaging, not a rewrite

Phase 1 being read-only is the important one. It lets the computed figure be compared against what agents actually entered, across thousands of historical files, before a single number is written automatically. That comparison is the cheapest validation available - it is how a ticket claiming 8% on a class EL AL files at 7% was found.

Non-negotiables

Rule	Reason
Money is integer minor units, never floats	A half-cent bias is invisible per ticket and material across a year
NIL and NO_RULE never merge	Both show \$0.00; only one is an answer
Reconcile against the cheque, not the commission	Otherwise every marked-up ticket reports wrongly
A tie is never broken arbitrarily	Two clauses matching equally returns AMBIGUOUS
Every figure cites its clause	The output is evidence in a dispute
The engine does no I/O	No database, no clock, no randomness - same ticket, same number
Unapproved clauses do not spend	A term nobody confirmed is exposure, not income

Open questions for Main St. Travel

Question	Impact
Is the commissionable basis the base fare alone, or base plus YQ/YR? Neither contract states it.	~\$35 per ticket on a long-haul fare
Is the host's point on a NET fare struck on the selling fare or the net fare before markup?	\$33.99 vs \$27.19 on one sampled ticket
Which booking classes count as Economy, Premium and Business for the cabin floor?	Decides a \$15 floor from a \$50 one
Is the tour code enforced in practice? No sampled ticket carries it.	Forfeits the commission on every published fare
Is the \$10 minimum fee on sub-\$10 commission automatic or discretionary?	\$10 per affected ticket
Statements are weekly, settlement is monthly - which is authoritative?	Whether a weekly shortfall is real or timing